

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №12.

Тема: «Ассоциативные контейнеры библиотеки STL»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

13	Задача 1 1. Контейнер - set 2. Тип элементов - double Задача 2 Тип элементов Pair (см. лабораторную работу №3). Задача 3 Параметризированный класс – Список (см. лабораторную работу №7)		
	Задание 3	Задание 4	Задание 5
	Найти максимальный элемент и добавить его в конец контейнера	Найти элементы ключами из заданного диапазона и удалить их из контейнера	К каждому элементу добавить среднее арифметическое контейнера.

Анализ задачи:

Задача 1.

- Создаем ассоциативный контейнер.
- Заполняем его элементами стандартного типа .
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Задача 2.

- Создаем ассоциативный контейнер.
- Заполняем его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегружаем необходимые операции.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Задача 3

- Создаем параметризированный класс, используя в качестве контейнера ассоциативный контейнер.
- Заполняем его элементами.
- Добавляем элементы в соответствии с заданием
- Удаляем элементы в соответствии с заданием.

Код:

```
#include<iostream>
```

```
#include<set>
```

```

#include<vector>

#include<cstdlib>

#include<ctime>

using namespace std;

// ===== KJIACC PAIR =====

class Pair {
private:
    double first;
    double second;
public:
    Pair() { first = 0; second = 0; }
    Pair(double f, double s) { first = f; second = s; }
    Pair(const Pair& p) { first = p.first; second = p.second; }
    ~Pair() {}

    double getFirst() const { return first; }
    double getSecond() const { return second; }
    void setFirst(double f) { first = f; }
    void setSecond(double s) { second = s; }

    Pair& operator=(const Pair& p) {
        if (&p == this) return *this;
        first = p.first;
        second = p.second;
        return *this;
    }

    bool operator>(const Pair& p) const {
        return (first > p.first) || (first == p.first && second > p.second);
    }

    bool operator<(const Pair& p) const {
        return (first < p.first) || (first == p.first && second < p.second);
    }
}

```

```

bool operator>=(const Pair& p) const { return !(*this < p); }
bool operator<=(const Pair& p) const { return !(*this > p); }
bool operator==(const Pair& p) const { return first == p.first && second == p.second; }

```

```

Pair operator+(const Pair& p) const {
    return Pair(first + p.first, second + p.second);
}

```

```

Pair operator/(const double& d) const {
    return Pair(first / d, second / d);
}

```

```

friend istream& operator>>(istream& in, Pair& p) {
    cout << "first? "; in >> p.first;
    cout << "second? "; in >> p.second;
    return in;
}

```

```

friend ostream& operator<<(ostream& out, const Pair& p) {
    out << "(" << p.first << ", " << p.second << ")";
    return out;
}
};

```

```

// ===== ЗАДАЧА 1: set<double> =====

```

```

typedef set<double> Set1;

```

```

Set1 make_set1(int n) {
    Set1 s;
    for (int i = 0; i < n; i++) {
        double a = rand() % 100 - 50;
        s.insert(a);
    }
    return s;
}

```

```

void print_set1(const Set1& s) {
    for (Set1::const_iterator it = s.begin(); it != s.end(); ++it)
        cout << *it << " ";
    cout << endl;
}

```

// Задание 3: Найти максимальный элемент и добавить его в конец

```

void task1_add_max(Set1& s) {
    if (s.empty()) return;
    double max = *s.rbegin();
    s.insert(max);
    cout << "Максимальный элемент (" << max << ") добавлен" << endl;
}

```

// Задание 4: Удалить элементы из диапазона

```

void task1_delete_range(Set1& s, double key1, double key2) {
    if (key1 > key2) swap(key1, key2);
    int count = 0;
    Set1::iterator it = s.lower_bound(key1);
    while (it != s.end() && *it <= key2) {
        it = s.erase(it);
        count++;
    }
    cout << "Удалено элементов: " << count << endl;
}

```

// Задание 5: Добавить среднее арифметическое к каждому элементу

```

void task1_add_average(Set1& s) {
    if (s.empty()) return;
    double sum = 0;
    for (Set1::const_iterator it = s.begin(); it != s.end(); ++it)
        sum += *it;
    double avg = sum / s.size();

    Set1 newSet;
    for (Set1::const_iterator it = s.begin(); it != s.end(); ++it)

```

```

        newSet.insert(*it + avg);
s = newSet;
cout << "Среднее арифметическое (" << avg << ") добавлено к каждому элементу" << endl;
}

```

```

void run_task1() {
    cout << "\n===== ЗАДАЧА 1: set<double> =====" << endl;
    Set1 s;
    int n;
    cout << "Введите количество элементов: ";
    cin >> n;
    s = make_set1(n);
    cout << "Исходный set: ";
    print_set1(s);

    int choice;
    cout << "\nВыберите задание:\n";
    cout << "1. Задание 3 - Добавить максимальный элемент\n";
    cout << "2. Задание 4 - Удалить элементы из диапазона\n";
    cout << "3. Задание 5 - Добавить среднее арифметическое\n";
    cout << "Ваш выбор: ";
    cin >> choice;

    switch (choice) {
    case 1:
        task1_add_max(s);
        cout << "Результат: ";
        print_set1(s);
        break;
    case 2: {
        double key1, key2;
        cout << "Введите начало диапазона: ";
        cin >> key1;
        cout << "Введите конец диапазона: ";
        cin >> key2;
        task1_delete_range(s, key1, key2);
    }
    }
}

```

```

        cout << "Результат: ";
        print_set1(s);
        break;
    }
    case 3:
        task1_add_average(s);
        cout << "Результат: ";
        print_set1(s);
        break;
    default:
        cout << "Неверный выбор!" << endl;
    }
}

// ===== ЗАДАЧА 2: set<Pair> =====

typedef set<Pair> Set2;

Set2 make_set2(int n) {
    Set2 s;
    for (int i = 0; i < n; i++) {
        double f = rand() % 50;
        double s_val = rand() % 50;
        s.insert(Pair(f, s_val));
    }
    return s;
}

void print_set2(const Set2& s) {
    for (Set2::const_iterator it = s.begin(); it != s.end(); ++it)
        cout << *it << " ";
    cout << endl;
}

void task2_add_max(Set2& s) {
    if (s.empty()) return;
    Pair max = *s.rbegin();

```

```

s.insert(max);

cout << "Максимальный элемент (" << max << ") добавлен" << endl;
}

```

```

void task2_delete_range(Set2& s, Pair key1, Pair key2) {
    if (key1 > key2) swap(key1, key2);
    int count = 0;
    Set2::iterator it = s.lower_bound(key1);
    while (it != s.end() && *it <= key2) {
        it = s.erase(it);
        count++;
    }
    cout << "Удалено элементов: " << count << endl;
}

```

```

void task2_add_average(Set2& s) {
    if (s.empty()) return;
    Pair sum(0, 0);
    for (Set2::const_iterator it = s.begin(); it != s.end(); ++it)
        sum = sum + *it;
    Pair avg = sum / s.size();

```

```

    Set2 newSet;
    for (Set2::const_iterator it = s.begin(); it != s.end(); ++it)
        newSet.insert(Pair(it->getFirst() + avg.getFirst(), it->getSecond() + avg.getSecond()));
    s = newSet;
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;
}

```

```

void run_task2() {
    cout << "\n===== ЗАДАЧА 2: set<Pair> =====" << endl;

    Set2 s;
    int n;

    cout << "Введите количество элементов: ";
    cin >> n;

    s = make_set2(n);

```

```
cout << "Исходный set: ";
```

```
print_set2(s);
```

```
int choice;
```

```
cout << "\nВыберите задание:\n";
```

```
cout << "1. Задание 3 - Добавить максимальный элемент\n";
```

```
cout << "2. Задание 4 - Удалить элементы из диапазона\n";
```

```
cout << "3. Задание 5 - Добавить среднее арифметическое\n";
```

```
cout << "Ваш выбор: ";
```

```
cin >> choice;
```

```
switch (choice) {
```

```
case 1:
```

```
    task2_add_max(s);
```

```
    cout << "Результат: ";
```

```
    print_set2(s);
```

```
    break;
```

```
case 2: {
```

```
    Pair key1, key2;
```

```
    cout << "Введите начало диапазона (Pair):\n";
```

```
    cin >> key1;
```

```
    cout << "Введите конец диапазона (Pair):\n";
```

```
    cin >> key2;
```

```
    task2_delete_range(s, key1, key2);
```

```
    cout << "Результат: ";
```

```
    print_set2(s);
```

```
    break;
```

```
}
```

```
case 3:
```

```
    task2_add_average(s);
```

```
    cout << "Результат: ";
```

```
    print_set2(s);
```

```
    break;
```

```
default:
```

```
    cout << "Неверный выбор!" << endl;
```

```
}
```

```
}
```

```
// ===== ЗАДАЧА 3: Параметризованный класс List =====
```

```
template<class T>
```

```
class List {
```

```
private:
```

```
    set<T> s;
```

```
public:
```

```
    List() {}
```

```
    List(int n) {
```

```
        T a;
```

```
        for (int i = 0; i < n; i++) {
```

```
            cin >> a;
```

```
            s.insert(a);
```

```
        }
```

```
    }
```

```
    void Print() {
```

```
        for (typename set<T>::const_iterator it = s.begin(); it != s.end(); ++it)
```

```
            cout << *it << " ";
```

```
        cout << endl;
```

```
    }
```

```
    void AddMaxToEnd() {
```

```
        if (s.empty()) return;
```

```
        T max = *s.rbegin();
```

```
        s.insert(max);
```

```
        cout << "Максимальный элемент добавлен" << endl;
```

```
    }
```

```
    void DeleteRange(T key1, T key2) {
```

```
        if (key1 > key2) swap(key1, key2);
```

```
        int count = 0;
```

```
        typename set<T>::iterator it = s.lower_bound(key1);
```

```
        while (it != s.end() && *it <= key2) {
```

```
            it = s.erase(it);
```

```

        count++;
    }
    cout << "Удалено элементов: " << count << endl;
}

void AddAverage() {
    if (s.empty()) return;
    T sum = *s.begin();
    int count = 1;
    for (typename set<T>::const_iterator it = ++s.begin(); it != s.end(); ++it) {
        sum = sum + *it;
        count++;
    }
    T avg = sum / count;

    set<T> newSet;
    for (typename set<T>::const_iterator it = s.begin(); it != s.end(); ++it)
        newSet.insert(*it + avg);
    s = newSet;
    cout << "Среднее арифметическое добавлено к каждому элементу" << endl;
}
};

```

```

void run_task3() {
    cout << "\n===== ЗАДАЧА 3: Параметризованный класс List =====" << endl;
    int n;
    cout << "Введите количество элементов: ";
    cin >> n;
    List<double> lst(n);
    cout << "Исходный список: ";
    lst.Print();

    int choice;
    cout << "\nВыберите задание:\n";
    cout << "1. Задание 3 - Добавить максимальный элемент\n";
    cout << "2. Задание 4 - Удалить элементы из диапазона\n";
}

```

```

cout << "3. Задание 5 - Добавить среднее арифметическое\n";
cout << "Ваш выбор: ";
cin >> choice;

switch (choice) {
case 1:
    lst.AddMaxToEnd();
    cout << "Результат: ";
    lst.Print();
    break;
case 2: {
    double key1, key2;
    cout << "Введите начало диапазона: ";
    cin >> key1;
    cout << "Введите конец диапазона: ";
    cin >> key2;
    lst.DeleteRange(key1, key2);
    cout << "Результат: ";
    lst.Print();
    break;
}
case 3:
    lst.AddAverage();
    cout << "Результат: ";
    lst.Print();
    break;
default:
    cout << "Неверный выбор!" << endl;
}
}

// ===== ГЛАВНОЕ МЕНЮ =====

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(0));

```

```

int choice;

do {

    cout << "\n===== \n";

    cout << "1. Задача 1 - set<double>\n";
    cout << "2. Задача 2 - set<Pair>\n";
    cout << "3. Задача 3 - Параметризованный класс List\n";
    cout << "0. Выход\n";

    cout << "===== \n";

    cout << "Выберите задачу: ";
    cin >> choice;


    switch (choice) {
    case 1:
        run_task1();
        break;
    case 2:
        run_task2();
        break;
    case 3:
        run_task3();
        break;
    case 0:
        cout << "Выход из программы..." << endl;
        break;
    default:
        cout << "Неверный выбор! Попробуйте снова." << endl;
    }


    if (choice != 0) {
        cout << "\nНажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
    }

} while (choice != 0);

```

```
system("pause");  
return 0;  
}
```

Результаты работы:

```
=====
1. Задача 1 - set<double>
2. Задача 2 - set<Pair>
3. Задача 3 - Параметризованный класс List
0. Выход
=====
Выберите задачу: 1

===== ЗАДАЧА 1: set<double> =====
Введите количество элементов: 6
Исходный set: -34 -11 2 19 26 48

Выберите задание:
1. Задание 3 - Добавить максимальный элемент
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 1
Максимальный элемент (48) добавлен
Результат: -34 -11 2 19 26 48

Нажмите Enter для продолжения...

=====
1. Задача 1 - set<double>
2. Задача 2 - set<Pair>
3. Задача 3 - Параметризованный класс List
0. Выход
=====
Выберите задачу: 2

===== ЗАДАЧА 2: set<Pair> =====
Введите количество элементов: 4
Исходный set: (15, 29) (40, 4) (42, 44) (45, 24)

Выберите задание:
1. Задание 3 - Добавить максимальный элемент
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 2
Введите начало диапазона (Pair):
first? 1
second? 2
Введите конец диапазона (Pair):
first? 2
second? 3
Удалено элементов: 0
Результат: (15, 29) (40, 4) (42, 44) (45, 24)
```

```
=====
1. Задача 1 - set<double>
2. Задача 2 - set<Pair>
3. Задача 3 - Параметризованный класс List
0. Выход
=====
Выберите задачу: 3

===== ЗАДАЧА 3: Параметризованный класс List =====
Введите количество элементов: 3
2
3
3
Исходный список: 2 3

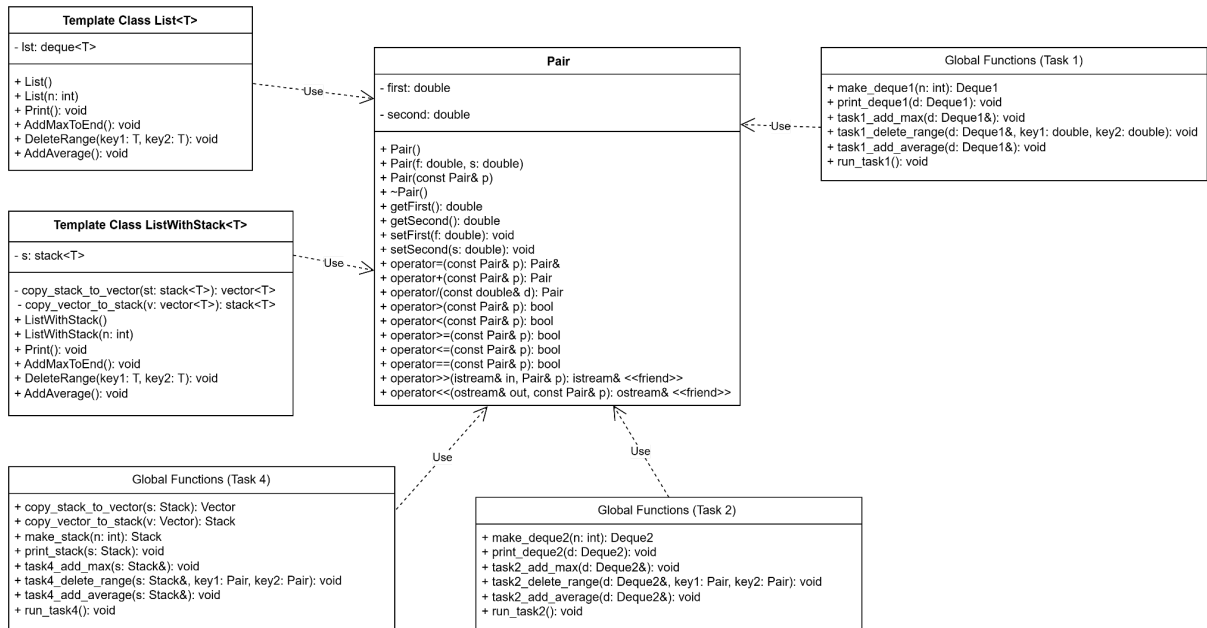
Выберите задание:
1. Задание 3 - Добавить максимальный элемент
2. Задание 4 - Удалить элементы из диапазона
3. Задание 5 - Добавить среднее арифметическое
Ваш выбор: 3
Среднее арифметическое добавлено к каждому элементу
Результат: 4.5 5.5

Нажмите Enter для продолжения...

=====
1. Задача 1 - set<double>
2. Задача 2 - set<Pair>
3. Задача 3 - Параметризованный класс List
0. Выход
=====
Выберите задачу: 0
Выход из программы...
Для продолжения нажмите любую клавишу . . .

C:\Users\Elena Lapteva\source\repos\main\x64\Debug\main.exe
```

UML:



Вопросы:

1. Ассоциативный контейнер содержит пары значений: ключ и отображенное значение. Зная ключ, можно получить доступ к значению.
2. map, multimap, set, multiset, bitset.
3. С помощью операции индексации [] (для map) или через итераторы (begin, end, rbegin, rend). Для multimap и multiset – только через итераторы.
4. insert, erase, clear, find, count, lower_bound, upper_bound, key_comp, value_comp, swap, empty, size, max_size.
5. map<int, float> m1; map<int, string> m2; map<int, Time> m3; также можно с инициализацией диапазоном или копированием.
6. По умолчанию упорядочены по возрастанию ключа с помощью less<Key>. Для обратного порядка нужно указать greater<Key> в параметре шаблона: map<int, T, greater<int>>.
7. Операции: ==, !=, <, <=, >, >=, оператор присваивания, оператор индексации [], методы insert, erase, clear, find, count, lower_bound, upper_bound, size, empty, swap.
- 8.

```

void add(map<int, T> &m, int key, T val) {
    m.insert(make_pair(key, val));
}
  
```

9.

```
void add(map<int, T> &m, int key, T val) {  
    m[key] = val;  
}
```

10.

```
void print(map<int, T> &m) {  
    for (auto it = m.begin(); it != m.end(); ++it) cout << it->first << " : " <<  
it->second << endl;  
}
```

11.

```
void print(map<int, T> &m) {  
    for (int i = 0; i < m.size(); ++i) cout << i << " : " << m[i] << endl;  
}
```

(но это работает только если ключи целые и идут подряд, в общем случае – неверно).

12. map хранит уникальные ключи, multimap допускает дубликаты. Для multimap не определена операция [].

13. set – ассоциативный контейнер, хранящий только ключи (без отдельных значений), упорядоченные по возрастанию. Элементы уникальны.

14. map хранит пары (ключ, значение), set хранит только ключи. В map ключ и значение могут быть разного типа, в set тип один.

15. set<int> s1; set<int> s2(a, a+5); set<int> s3(s2); где a – массив.

16. По умолчанию по возрастанию (less<T>). Для обратного порядка: set<T, greater<T>>.

17. Операции сравнения (==, !=, <, >, <=, >=), присваивание, insert, erase, clear, find, count, lower_bound, upper_bound, size, empty, swap, key_comp, value_comp.

18.

```
void add(set<T> &s, T val) {  
    s.insert(val);
```

```
}
```

19.

```
void print(set<T> &s) {  
    for (auto it = s.begin(); it != s.end(); ++it) cout << *it << " ";  
}
```

20. set хранит уникальные ключи, multiset допускает дубликаты. Для multiset метод count() может вернуть число больше 1, а find() возвращает первый найденный.